

Rescuing Performance from the Demo: Co-Designing Drum Gesture Mappings with a Percussionist

Appendix

Jordie Shier *

Centre for Digital Music
Queen Mary University of London
London, UK

Teresa Pelinski

Centre for Digital Music
Queen Mary University of London
London, UK

Charalampos Saitis

Centre for Digital Music
Queen Mary University of London
London, UK

Andrew Robertson

Ableton AG
Berlin, Germany

Andrew McPherson

Dyson School of Design Engineering
Imperial College London
London, UK

A TOOLKIT TECHNICAL DETAILS

The technical details of the final gesture mapping toolkit that we developed are presented here. Table 1 overviews the objects included in the toolkit.

A.1 Discrete Gesture Recognition

The discrete gesture recognition component of the system is event-based and utilizes onset detection to determine whether a drum has been struck at any given instance. In our work, we primarily focused on relatively simple classification between drum head and rim¹, which we found enough in a drum-kit context to create interesting performances when mapped to different pitches, synths, and other triggers. Figure 1 shows the Max4Live device, called `InputClassify`, supporting discrete gesture recognition.



Figure 1: M4L Device for discrete gesture recognition.

A.1.1 Method Overview

We used the method implemented in `DataKnot`² as a starting point, which uses `FluCoMa` (Tremblay et al., 2021) under-the-hood. The signal flow includes an onset detector to determine when a new event has occurred, audio feature extraction, and classification with a lightweight MLP. The onset detector used is the `FluCoMa AmpSlice` object, used throughout `DataKnot`, and is particularly effective for percussive signals while being lightweight and low-latency. This method reports onsets when an activation signal computed by taking the difference between to amplitude envelope followers—one fast reacting envelope and

one slow one—exceeds a specified threshold. Parameters for onset detection are included on the UI of the Max4Live device and include a sensitivity control, which maps to both on and off threshold values, a lock-out parameter which prevent re-triggering within a temporal window, and a noise floor parameter. These are all as implemented in `DataKnot`.

Audio feature extraction is performed on a window of 256 samples (≈ 5.8 ms at 44.1kHz) following a detected onset. Loudness, k-weighted, relative to full-scale (LKFS) (ITU, 2006) is extracted, which is normalized to a [0, 127] range for MIDI velocity. Spectral features are also optionally extracted and can be sent out for timbre remapping, discussed further in Section A.3.

The final input features used for classification were MFCCs. They include 13-coefficients computed on a 40-band Mel-band representation. The 13 coefficients start from the first coefficient, dropping the 0th which encodes signal amplitude. The minimum frequency is 200Hz and the maximum frequency is 12kHz. MFCC frames are summarized using the mean and standard deviation. These statistics are also computed on the first order derivative. All statistics are weighted using loudness. The resulting dimensionality of the feature vector is 52.

All data used for training and validating models was recorded during the co-design sessions and performed by the musician. Datasets are recorded using the `DatasetCreator` object, which performs onset detection and feature extraction using the approach outlined above. Classification MLPs can then be trained directly in Ableton using the `TrainClassifier` device, which utilizes `mlpclassifier~` from `FluCoMa`, or in Python³. During the study, Python was primarily used for training, which was useful for iterating model design and evaluation, which provided us with additional options for data pre-processing.

Prior to training, all features are normalized. Class imbalance is addressed in Python using the synthetic minority over-sampling technique (SMOTE) (Chawla et al., 2002), which balances classes by generating new data points for minority classes. For each data point, a random neighbour is selected and a new sample is

*j.m.shier@qmul.ac.uk

¹`DataKnot` and `Senory Percussion` allow for mapping of multiple different zones. This is possible in the toolkit, but is not something we explored in detail during the study.

²<https://github.com/rconstanzo/data-knot>

³Code available on the accompanying website:
<https://jordieshier.com/projects/aimc2026/>

Table 1: Objects in the percussive gesture mapping toolkit.

Name	Description
<i>Gesture Recognition</i>	
InputBrush	Input gesture recognition of continuous brushing gestures (uses Mel-band features).
InputBuzz	Input gesture recognition of buzz stroke rolls (uses onset activation signal).
InputClassify	Input gesture recognition of discrete gestures (drum hits).
<i>Parameter Mapping</i>	
MIDIReceive	Receives event messages from InputClassify and generates MIDI messages on a MIDI channel.
TimbreRemapRun	Receives audio features from InputClassify and predicts synthesizer parameters using a pre-trained neural network.
<i>Performance</i>	
Looper	Audio looper for creating layered performances.
NoteSeq	Step sequencer triggered from InputClassify .
PresetStep	Steps through synthesizer channels within an instrument rack, triggered from InputClassify .
<i>Utility</i>	
AutoGain	Automatically gain stages the input audio signal to a preset decibel level.
DatasetCreator	Records a dataset of features labelled by hit type.
TrainClassifier	Trains a neural network classifier using a dataset recorded by DatasetCreator . The trained model can then be loaded into InputClassify for inference.

generated in the direction of that neighbour by multiplying the difference between their feature vectors by a random number between 0 and 1, and then adding the result to the original sample.

An MLP classifier is then trained using the same approach used in FluCoMa, replicated in Python. This approach treats classification as a regression problem and optimizes the mean squared error (MSE). Output activations pass through a sigmoid non-linearity and loss computed on a one-hot encoded ground truth vector. The network is optimized using stochastic gradient descent.

In our study, we focused on simple classifications between the head and rim, although the Max4Live device can support multiclass classification of up to eight classes. A lightweight MLP with a single hidden layer of two neurons and a ReLU activation was sufficient for this prediction. Models were trained for a maximum of 1000 epochs with a learning rate of 0.001 and a momentum of 0.9. Early stopping is applied when loss on withheld 20% validation split doesn't decrease for 20 epochs.

FluCoMa's `mlpclassifier~` is used for inference in the Max4Live device, and each class can output one of three different messages, which are routed internally within the toolkit: 1) trigger only; 2) MIDI note and velocity; 3) spectral features.

A.2 Continuous Gesture Recognition

We focus on two different types of continuous gestures for detection: 1) gestures resulting in a continuously sustained sounds like brushing or "drawing" on a drum membrane with the tip of a stick; and 2) buzz stroke rolls or drags, which are distinguished from

more consistent strokes like single or double strokes. There are two different devices for each type of continuous gesture—**InputBrush** and **InputBuzz**—they differ only in their input audio representation, described below. In addition to gesture recognition, a continuous loudness feature is extracted, optionally gated by the continuous gesture prediction, and can be mapped within Ableton. Figure 2 shows the Max4Live device for continuous gesture recognition.



Figure 2: M4L Device for continuous gesture recognition. This device is **InputBuzz**; however, **InputBrush** shares the same interface design.

A.2.1 Input Representations

Brush and drawing gesture recognition use a Mel-spectrogram representation computed using an FFT with a size of 512 and a hop-size of 256 samples at a sampling rate of 48kHz. Each frame is windowed with a Hann window prior to the FFT. Mel-band magnitudes are then computed by multiplying the magnitude spectrum with triangular filters. We use 32 Mel-spaced filters with a minimum frequency of 1kHz and a maximum frequency of 20kHz. The continuous gestures are characterized by continuous noise, which we aim to capture with this representation. The resulting vector is then normalized by magnitude to preserve relative energy distribution across Mel-bands.

Mel-band magnitudes are computed from the FFT results as follow:

$$X'_k = \frac{4}{N} |X_k| \quad (1)$$

$$E = \frac{1}{2} \sum_k X'_k \quad (2)$$

$$\hat{m}_j = m_j \frac{E}{\sum_j m_j} \quad (3)$$

where $m_j = \sum_k X'_k f_{jk}$ and f_{jk} are the Mel filterbank weights for the j th Mel-band and k th FFT bin. The final Mel-bands are converted to a decibel scale.

Since the buzz stroke rolls are better characterized by temporal changes, we use an onset activation representation for these gestures. This activation signal is computed using the same method as the FluCoMa *AmpFeature* algorithm. First the input signal is high-pass filtered at 2kHz. This signal is then rectified, converted to a decibel scale, and floored:

$$v[n] = \max(-70, 20 \log_{10}(|x[n]|)) \quad (4)$$

This amplitude signal is then passed through two parallel envelope followers, a fast envelope follower and a slow one:

$$y[n] = y[n-1] + \alpha (v[n] - y[n-1]) \quad (5)$$

where

$$\alpha = \begin{cases} \alpha_{\text{up}} & \text{if } v[n] > y[n-1] \\ \alpha_{\text{down}} & \text{otherwise} \end{cases} \quad (6)$$

where the fast envelope follower has $\alpha_{\text{up}} = 1/3$ and $\alpha_{\text{down}} = 1/300$, and the slow envelope follower has $\alpha_{\text{up}} = \alpha_{\text{down}} = 1/2205$. These values are selected from the defaults in DataKnot. A difference envelope is then computed by subtracting the slow envelope from the fast envelope. These three envelopes are downsampled by a factor 64 and concatenated into a 3-dimensional time series.

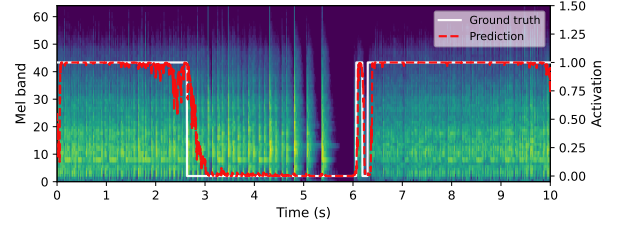
Both of these representations are normalized using values computed on a training dataset.

A.2.2 Neural Network Architecture and Training

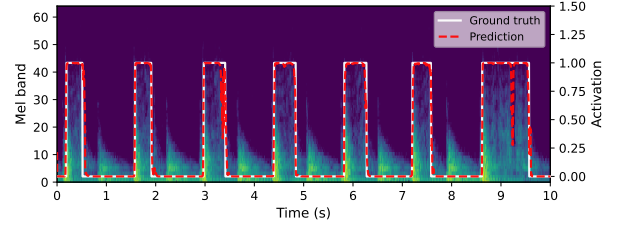
The recognition network utilizes a gated recurrent unit (GRU) (Cho et al., 2014) recurrent neural network with an architecture similar to the one used by Engel et al. (2020). Our version has a single GRU layer followed by an MLP. For our study, we used a GRU with a single hidden layer of size 16, and an MLP with three hidden layers, each with a size of 16 neurons. The MLP use ReLU activations on the hidden layers and the final 1D output has a sigmoid activation.

Training data was collected during the co-design sessions, and regions of continuous gestures manually labelled using Sonic Visualiser (Cannam et al., 2010). Models are trained in Python using binary cross entropy loss for 250 epochs with a batch size of 16 using an Adam optimizer with a learning rate of $1e-3$.

Figure 3 shows example outputs of two different models, one trained for buzz stroke roll recognition and the other for brush/drawing recognition.



(a) Continuous gesture example for a buzz stroke roll on a snare drum transitioning to a double stroke roll, then a buzz stroke roll starting again.



(b) Continuous gesture example for drawing gestures interleaved with hitting on a floor tom. The model ignores the hitting gestures and responds to the drawing.

Figure 3: Example segments from continuous gesture prediction. Each plot shows a spectrogram of the audio input, the ground truth labels, and the model prediction.

A.2.3 Real-time Implementation and Mapping

We used the RTNeural Plugin⁴ for Max/MSP, which builds on the RTNeural neural network inferencing library developed by Chowdhury (2021). Input features are extracted in Max/MSP using the same methods described above in Section A.2.1 at a frame rate of 750Hz (equivalent to a hop size of 64 samples). Neural network predictions occur at this frame rate. Optionally, the output stream of predictions is smoothed with an envelope follower with a parameterized exponential decay rate. Onset and offset thresholds are also parameterized on the UI of the Max4Live object. The object outputs a message similar to a sustain pedal MIDI CC value, which is high when in a continuous gesture and low otherwise.

Loudness is extracted continuously on the input signal, scaled to a $[-1, 1]$ range, and can be mapped to any parameter that can be modulated in Ableton. A highpass filter is applied before loudness computation with a parameterized cutoff frequency. Shaping is applied to the loudness feature:

$$y[n] = \text{clip}((\alpha_1 + \tanh((\alpha_0 + x[n]) \cdot \beta_0)) \cdot \beta_1) \quad (7)$$

where $\text{clip}(x) = \max(-1, \min(1, x))$, α_0 and α_1 are pre- and post-DC offsets, and β_0 and β_1 are pre- and post-gain factors, and $x[n]$ is the scaled loudness signal. To handle different ranges of loudness signals, we added an optional adaptive scaling that samples the loudness every 50ms and computes minimum and maximum values for scaling on the previous 20 values. The resulting signal is then smoothed using an expo-

⁴https://github.com/spluta/RTNeural_Plugin

nential envelope follower with controllable attack and release times.

Finally, there are options for how the feature behaves depending on whether a continuous gesture is detected. By default, the feature is always sent, regardless of the detected state. A *sustain gate* can be applied which allows active modulation only during a continuous gesture, and when a gesture stops the modulation can be set to drop to zero or held at the previous value. Attack and release times control how the signal behaves during transitions between continuous and non-continuous detected states.

A.3 Timbre Remapping

The timbre remapping component of the toolkit is based on the method described in (Shier et al., 2025), which we refer the reader to for full technical details. Briefly, an offline process generates a corpus of synthesizer presets paired with their associated audio features, which is used as training data for a neural network. At runtime, the network predicts synthesizer parameters from spectral audio features extracted at detected onsets. For this study, we adapted this approach for integration with Ableton Live as the **TimbreRemapRun** Max4Live device (see Table 1). Figure 4 shows the Max4Live device for timbre remapping inference. This object receives MIDI and spectral features from an **InputClassify** device, neural network inference is performed with the spectral features, producing synthesizer parameters, which are mapped to parameters in Ableton⁵, then the synth is triggered with the input MIDI event.



Figure 4: M4L Device for Timbre Remapping Inference

A.4 Performance Devices

The toolkit includes three performance devices to support musical structure within a live performance context.

The **Looper** device wraps the **karma~** looper⁶, and performs loop actions in response to a trigger from **InputClassify**. The looper here is a simple state machine that cycles from standby, to recording, to playback, and then back to standby. It was used in one track and enabled the musician to loop the output of a synthesizer and then play drums overtop. Figure 5 shows the Max4Live looper device.

NoteSeq is a performance device for creating a sequence of pitch values. The object is triggered via MIDI received from **InputClassify**. Optionally, the sequence can be stepped forward only when a specific MIDI note has arrived, enabling the musician to sit on

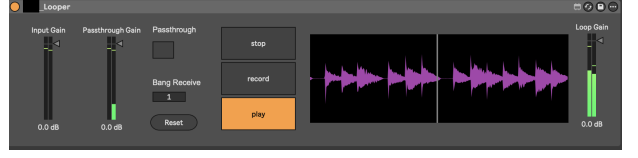


Figure 5: M4L Looper Device

a single note in the sequence until a gesture is received to change the loop.



Figure 6: M4L Note Sequencing Device

The final device is **PresetStep**, which was designed to allow for larger sonic changes like switching the synthesizer preset. This device operates as a state machine that steps when a trigger is received from an **InputClassify** device. The output state is configurable to have between 2 and 127 states, and is mappable within Ableton. Used in conjunction with an instrument rack with multiple chains, mapping the chain selector control enabled us to switch between different instrument chains from a drum trigger, which was useful for creating different musical sections within a piece. Figure 7 shows the device.

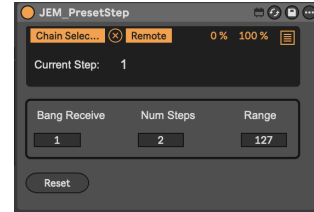


Figure 7: M4L Preset Step Device

REFERENCES

- Cannam, C., Landone, C., and Sandler, M. (2010). Sonic visualiser: an open source application for viewing, analysing, and annotating music audio files. In *Proceedings of the 18th ACM international conference on Multimedia*.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Cho, K., van Merriënboer, B., Bahdanau, D., and Bengio, Y. (2014). On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. In *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*.
- Chowdhury, J. (2021). RTNeural: Fast Neural Inference for Real-Time Systems. arXiv:2106.03037.

⁵There are eight mappable output parameters, designed to work with macros on an instrument rack.

⁶<https://rodrigoconstanzo.com/karma/>

- Engel, J., Hantrakul, L. H., Gu, C., and Roberts, A. (2020). DDSP: Differentiable digital signal processing. In *International Conference on Learning Representations*.
- ITU (2006). Algorithms to Measure Audio Programme Loudness and True-Peak Audio Level. *ITU-R BS.1770*. International Telecommunication Union.
- Shier, J., Constanzo, R., Saitis, C., Robertson, A., and McPherson, A. (2025). Designing Percussive Timbre Remappings: Negotiating Audio Representations and Evolving Parameter Spaces. In *International Conference on New Interfaces for Musical Expression*.
- Tremblay, P. A., Roma, G., and Green, O. (2021). Enabling Programmatic Data Mining as Musicking: The Fluid Corpus Manipulation Toolkit. *Computer Music Journal*, 45(2):9–23.